MADRE Agentic System

· Model-Agnostic · Delayed · Reasoning · Effort · [Beta v0.1]

Software Requirements Specification

Scvria Sofware

June 20, 2026

Contents

Operational Scenarios

1.1 Scenario Cards

Scenario / trigger	Module(s)	Runtime objects	Authority boundary	Expected records	Negative path
Foreground continuity / ordinary interaction	Core Module	Main Agent role, `ContextBundle`, retained interaction material	Answer only from permitted context and without making known material factual by implication.	Request and response `RuntimeEvent`; retained interaction material where configured.	Ambiguity, insufficient context, or risk creates clarification, block, or `ReasoningTask`, not an unsupported answer.
Delayed research / evidence required	Core Module and research module	`ReasoningPlan`, `ReasoningTask`, `ContextBundle`, optional `ModelAction`	Routing, data scope, and each action require applicable `PolicyDecision`.	Work lifecycle and evidence references in `RuntimeJournal`; `RuntimeEvent` evidence when inference runs.	Unverified or disallowed sources do not become accepted knowledge or authoritative response.
Knowledge handling / material selected for module use	Owning reasoning module	`KnowledgeRecord`; `KnowledgeCandidate` only for pending boundary change	Provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata govern use.	Knowledge creation/correction/detachment `RuntimeEvent` and related `PolicyDecision`.	Known material with unsuitable truth metadata is not presented as factual authority; saving it is not learning.
Specialized interaction / domain assistance requested	Core Module and selected domain module	retained interaction material, `ContextBundle`, module-owned `MADREAgent`, `ModelBinding`	A transferred or delegated scope does not transfer policy or knowledge authority.	Routing event, selected module, binding reference, context reference, and policy outcome.	Domain-specific output remains generated material until explicitly handled; it does not self-route or self-promote.
Workflow execution / reusable behavior invoked	Owning reasoning module	`Workflow`, `AgentRoutine`, `AgentAction`, `ReasoningTask`	Each executable action and external crossing is policy-gated with recovery expectation.	Action, work, policy, failure, and recovery `RuntimeEvent` history.	A hidden prompt chain or unauthorized side effect is blocked or authorization-required.
Evaluated improvement / repeated outcome suggests change	Owning reasoning module	`LearningCandidate`, evaluation evidence, optional `KnowledgeRecord` input	Promotion changes module behavior only after declared evaluation and promotion authority.	Candidate creation, evaluation, promotion/rejection, and rollback events.	generated material, conversation retention, or `KnowledgeRecord` storage alone never activates learning.

Developers may add models, internal actions, authorized external actions, projections, context policies, workflows, interfaces, and analysis helpers without rewriting the kernel, provided module ownership, `ModelBinding`, `PolicyDecision`, and `RuntimeJournal` trace obligations remain intact.

User Stories And Reference Use Cases

2.1 Baseline Purpose

The reference user stories preserve behavior pressure for the runtime class map. They are not implementation scripts. They show where module ownership, context minimization, delayed reasoning, workflow application, agent-routine execution, model-backed actions, knowledge handling, learning proposals, and trace evidence must remain distinguishable.

2.2 Representative User Stories

ID: UC-03 **Academic section with recent sources**

User Journey student

Epic: Research-grounded academic writing

User Voice: As a **student** I want *to write an academic section using recent evidence without leaking unrelated private context*

Acceptance:

- Delegates the academic task from `CoreModule` to `CollegeTasksModule`.
- Sends only a minimized, neutral research objective to `ResearchModule`.
- Checks retrieved material for source quality and injection risk.
- Returns evidence-backed draft material through the owning academic module.

ID: UC-04 **Private-context leakage prevention**

User Journey privacy-conscious user

Epic: Boundary-safe web research

User Voice: As a **privacy-conscious user** I want *to search externally without exposing sensitive personal details*

Acceptance:

- Detects private context mixed into a web-enabled research request.

- Separates the academic objective from sensitive user details.
- Records a boundary or policy outcome before delegation.
- Gives `ResearchModule` only a safe minimized query, or asks for confirmation.

ID: UC-05 **Assignment prompt injection**

User Journey student

Epic: Untrusted input handling

User Voice: As a **student** I want *to use assignment text as source material without letting hostile instructions execute*

Acceptance:

- Reviews pasted assignment text before treating it as task context.
- Classifies malicious instructions as source content, not authority.
- Blocks the harmful instruction from action, policy, or boundary influence.
- Records the security outcome while preserving usable academic content.

ID: UC-08 **LaTeX document compile recovery**

User Journey student

Epic: Artifact production and recovery

User Voice: As a **student** I want *to create a LaTeX assignment document and recover from compilation errors*

Acceptance:

- Applies a module workflow when the LaTeX assignment pattern is recognized.
- Invokes the LaTeX writing agent through concrete task execution.
- Materializes source, logs, and outputs as `ReasoningArtifact` records.
- Keeps compile/check actions bounded and records repair attempts.

ID: UC-12 **Generated draft becomes retained knowledge**

User Journey student

Epic: Knowledge lifecycle

User Voice: As a **student** I want *to remember a useful generated explanation for future assignments*

Acceptance:

- Treats the generated draft first as produced material, not active knowledge.
- Promotes or proposes it as a `KnowledgeRecord` only after explicit handling.
- Scopes the retained material as explanation style or accepted study explanation.
- Records the promotion, reclassification, or capture event.

ID: UC-13 **Repeated corrections produce a learning candidate**

User Journey user

Epic: Module-local learning

User Voice: As a **user** I want *to have repeated feedback improve future academic writing behavior*

Acceptance:

- Captures correction history as runtime evidence.
- Allows individual corrections to update relevant `KnowledgeRecord` entries.
- Forms a separate `LearningCandidate` for broader behavior change.
- Requires evaluation or promotion before the behavior change becomes active.

ID: UC-18 **Queued report under low resources**

User Journey user

Epic: Delayed work and inspectability

User Voice: As a **user** I want *to queue a large report when foreground completion is not practical*

Acceptance:

- Creates a `ReasoningPlan` with inspectable `ReasoningTask` objects.

- Responds immediately with queued-work status.
- Schedules execution as resources become available.
- Preserves lifecycle events and final artifacts for recovery and inspection.

ID: UC-26 **LaTeX worksheet with math and research**

User Journey student

Epic: Cross-module dependency chain

User Voice: As a **student** I want *to produce a worksheet that combines solved examples, recent references, and LaTeX output*

Acceptance:

- Lets `CollegeTasksModule` own the final academic deliverable.
- Delegates focused reasoning to `MathModule` and minimized research to `ResearchModule`.
- Integrates math results, research notes, and LaTeX artifacts into one plan.
- Records dependency links, validation steps, and final artifact delivery.

2.3 Cross-Use-Case Conclusions

- **KnowledgeRecord**:

  A `KnowledgeRecord` is broad retained module material. It may represent fact
  preference
  style
  procedure
  fiction
  hypothesis
  correction
  imported material
  or generated summary under provenance
  scope
  sensitivity
  intended use
  truth authority
  confidence
  and lifecycle metadata.

- **ReasoningArtifact**:

  A `ReasoningArtifact` is produced or materialized output. It may later inform knowledge handling but it is not active knowledge by default.

- **Generated material**:

  Generated material is ordinary risky material. It may be useful evidence or output

  but it cannot create authority

  policy

  routing

  knowledge

  action permission

  or learning by itself.

- **Boundary composition**:

  Executable validity comes from composed constraints across modules

  agents

  workflows

  routines

  actions

  context

  model use

  storage access

  and learning authority. This must be enforced by construction rather than hidden behind a loose policy flag.

- **Kernel routing**:

  The kernel resolves explicit valid targets or falls back to the Core Module. Semantic delegation remains module reasoning.

- **SystemAgent and TwinAgent**:

  `SystemAgent` is justified only as a strictly module-bound `ComplexAgent`. `TwinAgent` is justified by the checked-output invariant

  not by a particular model or thread layout.

- **Workflow and AgentRoutine**:

  `Workflow` is module-level planning structure that generates or refines `ReasoningTask` objects. `AgentRoutine` is agent-level executable compound capability that receives `AgentRequest` and returns `AgentResponse`.

- **Learning**:

  `LearningCandidate` proposes behavior improvement and remains inactive until evaluated promotion. Learning cannot widen privacy or action boundaries merely because it improves output quality.

System Engineering Baseline

3.1 Engineering Problem

MADRE addresses the engineering problem of coordinating model inference without allowing the model to become the system. The runtime must support foreground continuity, delayed reasoning, governed context, bounded internal actions, known material, learning, runtime traceability, and recovery while preserving local ownership of data and operational control.

The core difficulty is not only model accuracy. A useful local assistant must decide when an answer is safe in the foreground lane, when work must be delayed, which context may be exposed, which internal action may be invoked, which result requires verification, and how a mistaken or unsafe outcome can be inspected, corrected, or rolled back.

3.2 Stakeholders And Concerns

MADRE serves users who need private assistance, developers who need an extensible runtime, organizations that need governed local knowledge, and maintainers who must keep the system auditable and evolvable. Each stakeholder depends on the same architectural guarantees: model independence, local-first operation, explicit authority boundaries, understandable execution, and recoverable state.

The principal concerns are privacy, cost, autonomy control, model substitution, context minimization, data provenance, action safety, knowledge correction, learning control, operational continuity, and evidence for system behavior. A runtime that cannot expose those concerns as inspectable state cannot satisfy **MADRE** 's product objective, even if its conversational output appears fluent.

3.3 Requirement Domains

MADRE 's functional baseline is organized around runtime domains. The foreground lane domain preserves conversational flow, asks for clarification, and creates deeper work when the answer requires evidence or internal actions. The delayed reasoning domain records work durably, schedules it, verifies results, and returns outcomes without erasing the foreground interaction.

The context domain selects, minimizes, classifies, scopes, and revokes information before it reaches models or internal actions. The inference domain uses `ModelAction`, permitted `ModelBinding`, model-use profile, and replaceable model backend contracts. The internal action domain exposes declared side effects under `PolicyDecision` and `RuntimeEvent` recording. The knowledge domain separates retained interaction material, `KnowledgeRecord`, pending `KnowledgeCandidate`, untrusted source material, and evaluated `LearningCandidate`.

The learning domain keeps improvement proposals quarantined until evidence, evaluation, promotion, and rollback rules allow activation. The observability domain exposes sessions, `ReasoningTask` lifecycle, context bundles, `RuntimeEvent` evidence, actions, `PolicyDecision`, failures, metrics, and recovery paths. The deployment domain keeps ordinary local operation viable before optional remote integrations are considered.

3.4 Traceability Surface

Each significant product claim in **MADRE** must be visible as system behavior. A claim such as local-first operation corresponds to local execution paths, storage boundaries, remote-transfer controls, and inspection evidence. A claim such as model agnosticism corresponds to replaceable model-use profile and model backend contracts and tests that prevent provider-specific assumptions from entering the kernel.

The same trace applies to safety claims. Policy before action must appear as an enforceable boundary around internal actions, external calls, knowledge-boundary change, workflow execution, and learning promotion. Governed context must appear as provenance, classification, minimization, scope, revocation, and journaled trace. Delayed reasoning must appear as `ReasoningTask`, `RuntimeEvent` transitions, verification, result delivery, and recovery.

3.5 Quality Attributes

MADRE prioritizes data sovereignty, privacy, resource discipline, responsiveness, reliability, recoverability, maintainability, portability, auditability, and resistance to hallucination-driven action. These attributes are not independent decorations; they define whether the runtime fulfills its purpose.

Responsiveness is measured by preserving useful foreground conversation while heavier work is delayed. Reliability is measured by durable records, repeatable status, bounded retries, and visible failures. Maintainability is measured by the ability to replace models, internal actions, workflows, and interfaces without rewriting the kernel. Auditability is measured by the ability to explain what context, `RuntimeEvent` evidence, action, `PolicyDecision`, and verification path produced an outcome through `RuntimeJournal`.

3.6 Risk And Policy Boundaries

MADRE treats prompt injection, data leakage, excessive agency, unsafe action execution, poisoned knowledge, provider dependency, remote retention, unbounded consumption, credential exposure, and rollback gaps as architectural risks. The runtime must assume that generated material can be persuasive, wrong, incomplete, unsafe, or maliciously influenced by context.

Risk control is expressed through policy boundaries. Data classification governs what may be retrieved, stored, disclosed, or sent to a model. Autonomy level governs whether a result may remain a draft, update local state, invoke an internal action, contact an external service, or require stronger authorization. Recovery expectation governs whether the action must be reversible, snapshot-backed, compensatable, or blocked.

Requirements Baseline

4.1 Requirement Taxonomy

Prefix	Requirement family
FR	Functional behavior of foreground-lane conversation, delayed reasoning, context construction, inference, internal actions, knowledge, learning, runtime trace, and recovery.
NFR	Quality attributes such as responsiveness, reliability, recoverability, maintainability, portability, resource discipline, and auditability.
SEC	Security and trust controls, including prompt injection resistance, credential protection, action authority, and remote-transfer boundaries.
DATA	Data, knowledge, provenance, retention, deletion, correction, revocation, and quarantine requirements.
AI	`ModelAction`, model substitution, generated material handling, and verification requirements.
UX	User-visible states, clarification, authorization, progress, failure, correction, and result-delivery requirements.
OPS	Local deployment, storage, backup, diagnostics, migration, and operational recovery requirements.

4.2 Functional Requirements

ID	Requirement	Acceptance criteria	Verification
FR-001	The runtime shall preserve foreground-lane conversation while delaying evidence-heavy, risky, action-dependent, or broad-context work.	Immediate answers stay within safe known context; stronger work is recorded as delayed work.	Triage scenarios; D-001, D-004.
FR-002	The runtime shall represent delayed work as `ReasoningTask` objects with inspectable status and recovery path.	Work exposes queued, running, blocked, failed, completed, and recovered states with `RuntimeEvent` history in `RuntimeJournal`.	Work-lifecycle and recovery scenarios; D-001, D-003, D-008.
FR-003	The runtime shall build context bundles from sourced, classified, minimized, scoped, provenanced, and revocable material.	Context is validated or rejected before `ModelAction` or internal action use.	Context-governance scenarios; D-001, D-005, D-007.
FR-004	The runtime shall execute model-backed work through `ModelAction` constrained by `ModelBinding` and replaceable model-use profile/model backend contracts.	Every execution produces an inspectable `RuntimeEvent` evidence and keeps its generated material non-authoritative.	Inference substitution and output-boundary scenarios; D-001, D-006.
FR-005	The runtime shall expose executable behavior only as bounded internal actions or explicitly authorized external actions.	Internal actions remain policy-gated, while disallowed action requests are blocked or authorization-gated.	Action policy scenarios; D-007, D-009.
FR-006	The runtime shall keep `KnowledgeCandidate` and `LearningCandidate` semantics separate and inactive unless their respective promotion rules activate them.	Saving a `KnowledgeRecord` does not activate learning; evaluated behavior, routing, workflow, or inference-selection improvement remains a `LearningCandidate` until promoted.	Knowledge-boundary and learning-promotion scenarios; D-011.

ID	Requirement	Acceptance criteria	Verification
FR-007	The runtime shall expose local request entry and read-first inspection.	Local CLI request/status interaction and local console inspection expose system state without bypassing runtime authority.	Local access and inspection scenarios; D-010.
FR-008	The runtime shall represent recovery outcomes explicitly.	Correction, rollback, deletion/detachment, invalidation, and supersession are distinguishable and auditable.	Recovery readout scenarios; D-003, D-008.

4.3 Cross-Cutting Requirements

ID	Requirement	Rationale	Verification
NFR-001	Local useful operation shall remain possible without mandatory remote service dependency.	Data sovereignty, cost control, and resilience.	Local execution evidence; D-001, D-002, D-006.
NFR-002	Runtime state changes shall be inspectable through local read models.	Auditability and recoverability.	Inspection review; D-003, D-010.
NFR-003	Delayed work shall be bounded by status, resource, and failure discipline.	Prevents unbounded cost, compute, storage, and queue growth.	Resource/status scenarios; D-003, D-007, D-010.
SEC-001	Prompt or source material shall not become instruction, policy, or authority without explicit classification and boundary checks.	Source-as-instruction injection risk.	Adversarial context scenario; D-004, D-005, D-007.
SEC-002	A `PolicyDecision` shall explicitly represent allow, block, or authorization-required before action execution or promotion and be linked to `RuntimeEvent`.	Prevents hidden authority decisions and excessive agency.	Policy matrix review; D-007, D-009, D-011.

ID	Requirement	Rationale	Verification
SEC-003	External, destructive, credential, legal, financial, executable, or irreversible effects shall be blocked or separately authorized.	Preserves recoverability and explicit authority over critical action.	Critical-action negative paths; D-007, D-008, D-009.
DATA-001	A `KnowledgeRecord` shall carry provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata; it is not factual truth by default.	Knowledge governance and user control.	Non-authoritative known-material, correction, and revocation scenarios; D-005, D-008, D-011.
DATA-002	Active state and accountable `RuntimeEvent` history shall remain distinguishable.	Allows deletion/detachment without losing required trace history.	Recovery inspection scenarios; D-003, D-008.
AI-001	generated material shall remain generated material from an `RuntimeEvent` evidence, not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.	Hallucination and false-authority risk.	Output non-authority and non-candidate scenarios; D-001, D-006.
AI-002	Models and generated material shall not create or widen `PolicyDecision`, authorize actions, establish knowledge truth, alter behavior or routing, or activate learning.	Keeps runtime authority outside generated material.	Model-overreach and promotion-control scenarios; D-006, D-007, D-011.
UX-001	Users shall be able to distinguish immediate answers, queued work, running work, blocked work, failed work, completed work, and authorization requests.	Continuity and user control.	Interaction-state review; D-004, D-010, D-011.

ID	Requirement	Rationale	Verification
OPS-001	Local access surfaces shall submit requests and inspect `RuntimeJournal` status and trace without becoming runtime authority.	Supports operability while preserving governance.	Local access and inspection scenario; D-010.

Traceability Model

5.1 Traceability Method

Traceability in **MADRE** links the product claim, stakeholder concern, requirement, architecture obligation, validation method, and evidence status. The purpose is to keep evaluation anchored to the system thesis: local-first operation, model agnosticism, delayed reasoning, governed context, bounded internal actions, evaluated learning promotion, runtime traceability, and recovery.

5.2 Architecture Traceability Matrix

Claim	Concern	Requirement	Decision trace	Evidence family
Local-first operation	Data sovereignty, cost, resilience	NFR-001	D-001, D-002, D-010	Local request, local status, local journal trace, no mandatory remote service.
Model agnosticism	Provider independence	FR-004, AI-001	D-006	Replaceable model backend/model-use profile substitution plus generated material non-authority scenario.
Delayed reasoning	Responsiveness and quality	FR-001, FR-002	D-001, D-003, D-004	Foreground triage plus durable delayed-work lifecycle.
Governed context and known material	Privacy and correctness	FR-003, DATA-001	D-005, D-007	Context acceptance, source-as-data, non-authoritative `KnowledgeRecord` use, truth-metadata correction, and revocation scenario.

Claim	Concern	Requirement	Decision trace	Evidence family
Bounded internal actions	Safety and recoverability	FR-005, SEC-001	D-007, D-009	Policy-gated internal action and blocked unsafe external action attempts.
Learning promotion control	Controlled evolution	FR-006	D-011	`LearningCandidate` evaluation and inspection without active promotion; `KnowledgeRecord` save is not treated as learning.
Runtime trace and recovery	Accountability and correction	FR-008, NFR-002, DATA-002	D-003, D-008, D-010	`RuntimeJournal`/`Runtime` status, correction, invalidation, supersession, rollback, and deletion/detachment trace.

5.3 Integrated Architecture Trace

Step	Required behavior	Decision trace	Negative evidence
Local entry	Request enters through local access surface.	D-001, D-010	Internal-only execution is not enough to prove inspection.
Foreground triage	Foreground answers only safe known context, clarifies, blocks, or creates delayed work.	D-004, D-007	Evidence-heavy request is not answered as if already known.
Durable work	Delayed work is represented by `ReasoningTask` with `RuntimeEvent` status history.	D-003	Work is not lost or silently overwritten.

Step	Required behavior	Decision trace	Negative evidence
Governed context	Context is sourced, classified, minimized, scoped, provenanced, revocable, and linked to journal trace.	D-005, D-007	Sensitive, unrelated, revoked, or incorrectly truth-classified material is excluded or blocked.
Inference boundary	`ModelAction` uses model backend, creates `RuntimeEvent` evidence, and returns non-authoritative generated material.	D-006	Generated material does not become authority or a candidate by itself.
Bounded internal action	Typed runtime-defined internal actions are policy-gated and recorded.	D-009	Remote, host-command, filesystem, or mutation actions require stronger authority boundaries.
Knowledge and learning handling	`KnowledgeCandidate` covers pending knowledge-boundary change; `LearningCandidate` covers evaluated improvement.	D-011	`KnowledgeRecord` storage does not activate learning and candidates cannot promote themselves.
Inspection and recovery	Status, trace, failure, correction, rollback, invalidation, supersession, and deletion/detachment are visible through `RuntimeJournal`.	D-008, D-010	Active state changes do not erase accountable history.

Requirements Catalogue

ID	Requirement summary	Decision trace
FR-001	Preserve foreground-lane conversation while delaying work that requires evidence, verification, internal actions, or stronger context.	D-001, D-004.
FR-002	Record delayed reasoning as `ReasoningTask` with `RuntimeEvent` lifecycle and recovery path through `RuntimeJournal`.	D-001, D-003, D-008.
FR-003	Construct context bundles from sourced, classified, minimized, scoped, provenanced, and revocable information.	D-001, D-005, D-007.
FR-004	Execute model-backed work through `ModelAction`, allowed `ModelBinding`, and replaceable model backend, producing `RuntimeEvent` evidence and non-authoritative generated material.	D-001, D-006.
FR-005	Expose executable behavior only as declared, bounded, traceable internal actions or explicitly authorized external actions.	D-007, D-009.
FR-006	Keep pending `KnowledgeCandidate` change separate from evaluated `LearningCandidate` improvement and require promotion for each.	D-011.
FR-007	Expose local request entry and read-first inspection.	D-010.
FR-008	Represent recovery outcomes explicitly.	D-003, D-008.
SEC-001	Treat prompt/source material as data rather than instruction; treat generated material as produced material rather than authority or candidate by itself.	D-004, D-005, D-007.
OPS-001	Keep local access surfaces governed by runtime authority.	D-010.

Traceability Matrix

Requirement	Concern	Architecture element	Evidence family
FR-001	Responsiveness and user control	Complex Agent foreground lane and delayed reasoning split	Interaction and latency scenarios.
FR-002	Recoverability and traceability	`ReasoningTask`, `RuntimeEvent`, and `RuntimeJournal`	Recovery and status scenarios.
FR-003	Privacy and correctness	Governed context bundle	Context isolation scenarios.
FR-004	Provider independence	`ModelAction`, model backend, `RuntimeEvent` evidence, and generated material	Inference substitution and output-boundary scenarios.
FR-005	Action safety	Agent Action boundary, action contract, and external action policy	Allowed-action and blocked-action scenarios.
FR-006	Controlled evolution	Separate knowledge-change and learning-promotion candidates	Knowledge-boundary and learning-promotion review.
FR-007	Local operability	CLI plus read-first local console	Local access and inspection scenario.
FR-008	Recoverability	`RuntimeEvent` history and recovery semantics	Recovery trace.